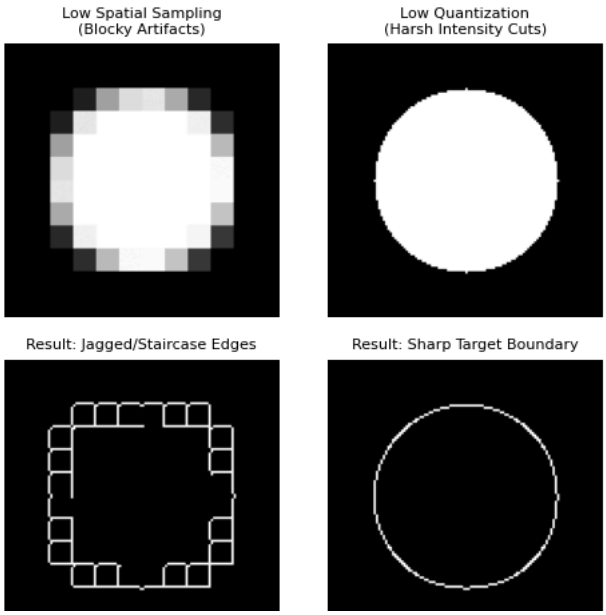
```
import cv2                                                  # Import OpenCV for image processing algorithms
import numpy as np                                          # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt                             # Import Matplotlib to visualize the output on screen

# ======================================================================
# STEP 1: CREATE A HIGH-QUALITY CIRCULAR OBJECT TARGET
# ======================================================================
# Generate a clean 120x120 grayscale scene containing a smooth round object
base_scene = np.zeros((120, 120), dtype=np.uint8)
cv2.circle(base_scene, (60, 60), 40, 255, -1)

# ======================================================================
# STEP 2: DEGRADE QUALITY METHOD A - LOW SPATIAL SAMPLING (Pixel Drop)
# ======================================================================
# Drop the pixel count down to a tiny 12x12 grid, then scale it back up
downsampled = cv2.resize(base_scene, (12, 12), interpolation=cv2.INTER_AREA)
low_sampling = cv2.resize(downsampled, (120, 120), interpolation=cv2.INTER_NEAREST)

# ======================================================================
# STEP 3: DEGRADE QUALITY METHOD B - LOW GRAY QUANTIZATION (Bit Drop)
# ======================================================================
# Reduce color options to a 1-bit binary state (only pure black or pure white)
_, low_quantization = cv2.threshold(base_scene, 127, 255, cv2.THRESH_BINARY)

# ======================================================================
# STEP 4: EXTRACT EDGES TO SHOW IMPACT ON PERCEPTION DATA
# ======================================================================
# Run Canny Edge Detection to see what structural details survive
edge_sampling = cv2.Canny(low_sampling, 100, 200)
edge_quantization = cv2.Canny(low_quantization, 100, 200)

# ======================================================================
# STEP 5: VISUALIZE RESOLUTION ANALYSIS SIDE-BY-SIDE (COMPACT)
# ======================================================================
fig, axes = plt.subplots(2, 2, figsize=(5, 5))

# Row 1: The degraded input frames
axes[0, 0].imshow(low_sampling, cmap='gray')
axes[0, 0].set_title('Low Spatial Sampling\n(Blocky Artifacts)', fontsize=8)
axes[0, 0].axis('off')

axes[0, 1].imshow(low_quantization, cmap='gray')
axes[0, 1].set_title('Low Quantization\n(Harsh Intensity Cuts)', fontsize=8)
axes[0, 1].axis('off')

# Row 2: The output features extracted by the system
axes[1, 0].imshow(edge_sampling, cmap='gray')
axes[1, 0].set_title('Result: Jagged/Staircase Edges', fontsize=8)
axes[1, 0].axis('off')

axes[1, 1].imshow(edge_quantization, cmap='gray')
axes[1, 1].set_title('Result: Sharp Target Boundary', fontsize=8)
axes[1, 1].axis('off')

plt.tight_layout()
plt.show()
```


Low Spatial Sampling
(Blocky Artifacts)
Low Quantization
(Harsh Intensity Cuts)
Result: Jagged/Staircase Edges
Result: Sharp Target Boundary